typesense

Natural Language Search

Copy Markdown

One of the most powerful capabilities of Large Language Models (LLMs) is their ability to turn natural language into structured data. In this guide, we will learn how to make use of this capability to understand a user's search query and convert it into a structured Typesense search query.

Use-case

Let's take an example of a public cars dataset . Using Google's Gemini LLM along with Typesense we can support natural language queries like the following:

- `A honda or BMW with at least 200hp, rear-wheel drive, from 20K to 50K, must be newer than 2014`
- `Show me the most powerful car you have`
- `High performance Italian cars, above 700hp`
- `I don't know how to drive a manual`

Notice how in some queries there might be multiple criteria mentioned, and in some cases the keyword itself might not be present in the dataset.

Here's a sample record from this dataset for context:

json


```
{
  "city_mpg": 13,
  "driven_wheels": "rear wheel drive",
  "engine_cylinders": 8,
  "engine_fuel_type": "premium unleaded (recommended)",
  "engine_hp": 707,
  "highway_mpg": 22,
  "id": "1480",
  "make": "Dodge",
  "market_category": ["Factory Tuner", "High-Performance"],
  "model": "Charger",
  "msrp": 65945,
```

```
    "vehicle_size": "Large",
    "vehicle_style": "Sedan",
    "year": 2017
}
```

Data flow

The key idea is this:

1. Take the natural language query that the user types in
2. Send it to the LLM with specific instructions on how to convert it into a Typesense search query with the `filter_by`, `sort_by` and `q` search parameters
3. Execute a query in Typesense with those search parameters returned by the LLM and return the results

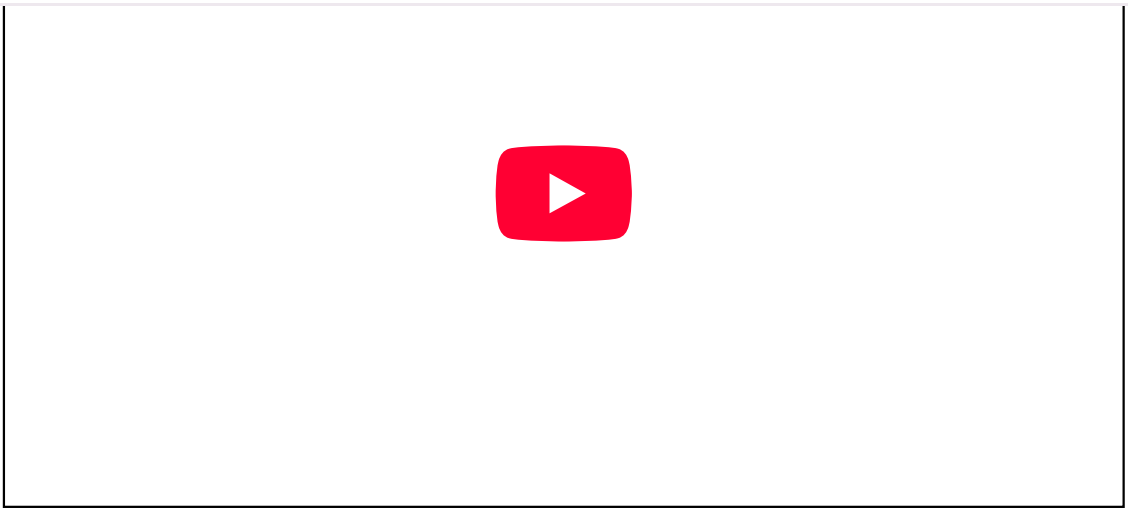
We're essentially doing something similar to Text-to-SQL, except that we're now doing Text-to-Typesense-Query, running the query and returning results.

This seemingly simple concept helps build powerful natural language search experiences. The trick though with LLMs is to refine the prompt well-enough that it consistently produces a good translation of the text into valid Typesense syntax.

> **Natural Language Search is now built-in to Typesense** v29.0+
>
> As of v29.0 of Typesense, the Natural Language Search feature is now built-in to Typesense, where you provide just your LLM API Keys and Typesense will take care of all the necessary orchestration described above to convert users' queries into structured Typesense queries. Read the docs on how to use this built-in feature.
>
> The rest of this guide talks about how to build something similar outside of Typesense, which used to be the older way of implementing Natural Language Search, before this feature was built-in in v29.0.

Live Demo

Here's a video of what we'll be building in this guide:



You can also play around with it live here: https://natural-language-search-cars-genkit.typesense.org/

Let's now see how to build this application end-to-end.

Setting up the project

We will be using Next.js and Genkit which is a framework that makes it really easy to add generative AI in our applications.

Follow the instructions in Genkit's documentation to learn how to initialize Genkit in a Next.js app.

Next, let's install the Typesense client into our app:

```
npm i typesense@next
```

The dataset we will use can be downloaded from Github .

Initializing Typesense client

We will need two separate Typesense API keys:

- A search-only API key for use on the front end
- A backend API key with write access

will give you a set of hostnames and API keys to use.

JavaScript

```
import Typesense from 'typesense'
/*
 *  Our JavaScript client library works on both the server and the browser.
 *  When using the library on the browser, please be sure to use the
 *  search-only API Key rather than an admin API key since the latter
 *  has write access to Typesense and you don't want to expose that.
 */
export const typesense = ({ isServer = false } = {}) =>
  new Typesense.Client({
    apiKey:
      (isServer ? process.env.TYPESENSE_ADMIN_API_KEY : process.env.NEXT_PUBLIC_TYPESENSE_S
    nodes: [
      {
        url: process.env.NEXT_PUBLIC_TYPESENSE_URL || 'http://localhost:8108',
      },
    ],
    connectionTimeoutSeconds: 5,
  })
```

Create the Typesense collection

We'll use the following schema to create a Typesense Collection and import our cars dataset into it:

JavaScript

```
typesense({ isServer: true })
  .collections()
  .create({
    name: 'cars',
    fields: [
      { name: 'make', type: 'string', facet: true },
      { name: 'model', type: 'string', facet: true },
      { name: 'year', type: 'int32' },
```

```
      { name: 'transmission_type', type: 'string', facet: true },
      { name: 'driven_wheels', type: 'string', facet: true },
      { name: 'number_of_doors', type: 'int32' },
      { name: 'market_category', type: 'string[]', facet: true },
      { name: 'vehicle_size', type: 'string', facet: true },
      { name: 'vehicle_style', type: 'string', facet: true },
      { name: 'highway_mpg', type: 'int32' },
      { name: 'city_mpg', type: 'int32' },
      { name: 'popularity', type: 'int32' },
      { name: 'msrp', type: 'int32' },
    ],
  })
```

We're now ready to index the dataset into the collection we just created:

JavaScript

```
let fs = require('fs/promises')

const carsInJsonl = await fs.readFile('cars.jsonl')
// IMPORTANT: Be sure to increase connectionTimeoutSeconds to at least 5 minutes or more fo
//  when instantiating the client
typesense({ isServer: true }).collections('cars').documents().import(carsInJsonl)
```

Writing the prompt

Our goal is to translate a natural language query e.g. `'Latest Ford under 40K$'` into Typesense's query format:

json

```
{
  "filter_by": "make:Ford && msrp:<40000",
  "sort_by": "year:desc"
}
```

In Genkit, the model output schema is defined using Zod.

```
import { z } from 'genkit'

const TypesenseQuerySchema = z
  .object({
    query: z.string().describe('a full-text search query'),
    filter_by: z.string().describe('a filter query in Typesense format'),
    sort_by: z.string().describe('a sorting query in Typesense format'),
  })
  .partial()
```

We can make the LLM ouput conform to our `TypesenseQuerySchema` by specifying it in `generate()`:

JavaScript

```
const { output } = await ai.generate({
  model: gemini15Flash,
  output: { schema: TypesenseQuerySchema },
  system: // prettier-ignore
      `You are assisting a user in searching for cars. Convert their query into the appropr

### Typesense Query Syntax ###

## Filtering ##

Matching values: The syntax is {fieldName} follow by a match operator : and a string value
- model:prius
- make:[BMW,Nissan] returns cars that are manufactured by BMW OR Nissan.

Numeric Filters: Use :[min..max] for ranges, or comparison operators like :>, :<, :>=, :<=,
 - year:[2000..2020]
 - highway_mpg:>40
 - msrp:=30000

Multiple Conditions: Separate conditions with &&. Examples:
 - num_employees:>100 && country:[USA,UK]
 - categories:=Shoes && categories:=Outdoor

OR Conditions Across Fields: Use || only for different fields. Examples:
 - vehicle_size:Large || vehicle_style:Wagon
 - (vehicle_size:Large || vehicle_style:Wagon) && year:>2010
```

```
 - make:!=[Nissan,BMW]

 If the same field is used for filtering multiple values in an || (OR) operation, then use
\`make:BMW || make:Honda || make:Ford\`
should be simplified as:
\`make:[BMW, Honda, Ford]\`

If any string values have parentheses, surround the value with backticks to escape them.

For eg, if a field has the value "premium unleaded (required)", and you need to use it in a

- fuel_type:\`premium unleaded (required)\`
- fuel_type!:\`premium unleaded (required)\`

## Sorting ##

You can only sort maximum 3 sort fields at a time. The syntax is {fieldName}: follow by asc
 - msrp:desc
 - year:asc,city_mpg:desc

Sorting hints:
  - When a user says something like "good mileage", sort by highway_mpg or/and city_mpg.
  - When a user says something like "powerful", sort by engine_hp.
  - When a user says something like "latest", sort by year.

### Query ###
Include query only if both filter_by and sort_by are inadequate.

## Car properties ##

| Name | Data Type | Filter | Sort | Enum Values | Description |
|------|-----------|--------|------|-------------|-------------|
${await getCachedCollectionProperties()}

### Output Instructions ###
Provide the valid JSON with the correct filter and sorting format, only include fields with

  prompt: //prettier-ignore
`### User-Supplied Query ###
${query}`,
})
```

Dynamic Prompt based on the Schema

Notice the `getCachedCollectionProperties()` function in the prompt above.

That function essentially converts the Typesense collection schema into a tabular format with a list of field names and sample enum values in each field. We're using a markdown format to help the LLM recognize these field values in the query and convert them into appropriate field filters.

Here's an example of what the output of that function could look like:

md

```
## Car properties

| Name               | Data Type | Filter | Sort | Enum Values
| ------------------ | --------- | ------ | ---- | -----------------------------------
| year               | int32     | Yes    | Yes  |
| engine_hp          | float     | Yes    | Yes  |
| engine_cylinders   | int32     | Yes    | Yes  |
| number_of_doors    | int32     | Yes    | Yes  |
| highway_mpg        | int32     | Yes    | Yes  |
| city_mpg           | int32     | Yes    | Yes  |
| popularity         | int32     | Yes    | Yes  |
| msrp               | int32     | Yes    | Yes  |
| make               | string    | Yes    | No   | Chevrolet, Ford, Dodge, Mercedes-Be
| model              | string    | Yes    | No   | 911, F-150, Tundra, E-Class, Silver
| engine_fuel_type   | string    | Yes    | No   | regular unleaded, premium unleaded
| transmission_type  | string    | Yes    | No   | AUTOMATIC, MANUAL, AUTOMATED_MANUAL
| driven_wheels      | string    | Yes    | No   | front wheel drive, rear wheel drive
| market_category    | string[]  | Yes    | No   | Luxury, Performance, High-Performar
| vehicle_size       | string    | Yes    | No   | Compact, Midsize, Large
| vehicle_style      | string    | Yes    | No   | Sedan, 4dr SUV, Coupe, Convertible,
```

Here's how that function looks:

JavaScript

```
async function getCollectionProperties() {
  const collection = await typesense({ isServer: true }).collections('cars').retrieve()
  const facetableFields = []
  const rows = []
```

```
      facetableFields.push(field)
    } else {
      const { name, type, sort } = field
      rows.push(
        // prettier-ignore
        `|${name}|${type}|Yes|${booleanToYesNo(sort)}||${collection.metadata?.[name] || ''}
      )
    }
  })

  const facetValues = await typesense()
    .collections('cars')
    .documents()
    .search({
      q: '*',
      facet_by: facetableFields?.map(({ name }) => name).join(','),
      max_facet_values: MAX_FACET_VALUES + 1, // plus 1 so we can check if any fields excee
    })

  const facetableRows = facetableFields?.map(({ type, name, sort }, i) => {
    const counts = facetValues.facet_counts?.[i].counts
    const exceedMaxNumValues =
      counts && counts?.length > MAX_FACET_VALUES ? 'There are more enum values for this fi
    const enums = counts?.map(item => item.value).join(', ')
    // prettier-ignore
    return `|${name}|${type}|Yes|${booleanToYesNo(sort)}|${enums}|${collection.metadata?.[n
    }${exceedMaxNumValues}|`;
  })
  return rows.concat(facetableRows).join('\n')
}

function booleanToYesNo(bool: boolean | null | undefined) {
  return bool ? 'Yes' : 'No'
}
```

For facet enabled fields, we can supply facet values to the LLM via the `Enum Values` column by making a search request with `q: "*", facet_by: field`. When a collection has too many facet values to fit in the prompt, the number of values returned can be limited using the `max_facet_values` parameter.

JavaScript

```
await typesense.collections('cars').update({
  metadata: {
    msrp: 'in USD',
  },
})
```

Since fetching the collection properties everytime the user make a search request is expensive, we will cache its response using Nextjs `unstable_cache` .

JavaScript

```
import { unstable_cache } from 'next/cache'

const getCachedCollectionProperties = unstable_cache(async () => await getCollectionPropert
  tags: ['getCollectionProperties'],
  revalidate: false, // Since the Typesense data for this demo is static, we will cache the
})
```

Integrate with Typesense

Let's now integrate our dynamic prompt into our application:

JavaScript

```
'use server'
import { genkit, z } from 'genkit'
import { gemini15Flash, googleAI } from '@genkit-ai/googleai'
import { TypesenseQuerySchema } from '@/schemas/typesense'

const ai = genkit({
  plugins: [googleAI()],
  model: gemini15Flash,
})

const generateTypesenseQuery = ai.defineFlow(
```

```
    outputSchema: TypesenseQuerySchema,
  },
  async query => {
    const { output } = await ai.generate({
      model: gemini15Flash,
      output: { schema: TypesenseQuerySchema },
      system: `...`,
      prompt: //prettier-ignore
`### User-Supplied Query ###
${query}`,
    })

    if (output == null) {
      throw new Error("Response doesn't satisfy schema.")
    }
    return output
  },
)
```

Finally, we can call the server action and use its response to make a search request to Typesense.

JavaScript

```
async function getCars(q: string) {
  const generatedQ = await callGenerateTypesenseQuery(q)

  const params = {
    q: generatedQ.query || '*',
    filter_by: generatedQ.filter_by || '',
    sort_by: generatedQ.sort_by || '',
  }

  const searchResponse = await typesense()
    .collections('cars')
    .documents()
    .search({
      ...params,
      query_by: 'make,model,market_category',
      per_page: 12,
    })

  console.log(searchResponse)
```

That's it! Our users can now use natural language to search for cars and the intended filters will automatically be applied!

Keep in mind that LLMs may occasionally misunderstand queries or generate Typesense queries that are invalid. In such cases, tweaking the prompt to handle specific edge cases or incorporating fallback logic can ensure better results over time.

You can find the full source code of the demo application on GitHub and a live demo here .

> **Natural Language Search is now built-in to Typesense** v29.0+
>
> As of v29.0 of Typesense, the Natural Language Search feature is now built-in to Typesense, where you provide just your LLM API Keys and Typesense will take care of all the necessary orchestration described above to convert users' queries into structured Typesense queries. Read the docs on how to use this built-in feature.

Edit page

Last Updated: 6/18/2025, 11:12:22 PM

← Query Suggestions

Boolean Search with Tags →